Airflow ML Pipeline - Kurulum ve Kullanım Kılavuzu

Bu dokümantasyon, EV Fleet Management sistemi için Airflow ML pipeline'ının kurulumunu, yapılandırmasını ve kullanımını açıklar.

Genel Bakış

Bu pipeline şunları yapar:

```
MariaDB (Anlık Telemetri) → TimescaleDB (İşlenmiş Segmentler) → H2O AutoML
(Model Eğitimi) → MLflow (Model Registry) → FastAPI (Tahmin Servisi)
```

Ana Bileşenler

Servis	Port	Açıklama
Airflow Webserver	8085	DAG yönetimi ve izleme
Airflow Scheduler	-	Görev zamanlayıcı
PostgreSQL	5432	Airflow metadata DB
TimescaleDB	5433	ML veri deposu
MLflow	5000	Model registry ve experiment tracking
FastAPI	8010	Tahmin API servisi

Gereksinimler

Yazılım

- **Docker Desktop** (Windows/Mac) veya **Docker Engine** (Linux)
- **Docker Compose** v2+
- **Git** (opsiyonel)

Donanım (Minimum)

- RAM: 8 GB (16 GB önerilir)
- Disk: 10 GB boş alan
- CPU: 4 çekirdek

Ağ

- MariaDB erişimi (host.docker.internal:3306)
- İnternet bağlantısı (Docker image'ları için)

Kurulum

Adım 1: Proje Dosyalarını Kopyalayın

```
# Projeyi server'a kopyalayın
scp -r Fleet_v1.0/app/airflow_docker user@server:/path/to/destination/
```

Adım 2: Ortam Değişkenlerini Ayarlayın

`airflow_docker/.env` dosyasını düzenleyin:

```
# Airflow
AIRFLOW_UID=50000
AIRFLOW__CORE__FERNET_KEY=your-fernet-key-here

# MariaDB (Ana veri kaynağı)
# NOT: Aşağıdaki bilgileri kendi MariaDB bağlantınıza göre değiştirin
# DAG dosyasında (dags/ev_full_pipeline.py) MARIADB_URI'yi güncelleyin
```

Adım 3: MariaDB Bağlantısını Yapılandırın

`dags/ev_full_pipeline.py` dosyasında (satır ~22):

```
# CONNECTION - Kendi MariaDB bilgilerinizi girin
MARIADB_URI =
"mysql+mysqldb://root:YOUR_PASSWORD@host.docker.internal:3306/fleetmanagement
db"
```

> **Önemli:** `host.docker.internal` Windows/Mac'te çalışır. Linux'ta host IP'sini kullanın.

Adım 4: Docker Image'larını Oluşturun

```
cd airflow_docker
docker-compose build
```

Adım 5: Veritabanlarını Başlatın

```
# İlk olarak sadece veritabanlarını başlatın
docker-compose up -d postgres timescaledb

# 30 saniye bekleyin (veritabanları hazırlanıyor)
sleep 30
```

Adım 6: Model Metadata Tablolarını Oluşturun

```
# Windows PowerShell
Get-Content init_model_tables.sql | docker exec -i timescaledb psql -U root -
d ev_telemetry

# Linux
docker exec -i timescaledb psql -U root -d ev_telemetry <
init_model_tables.sql
```

Adım 7: Tüm Servisleri Başlatın
```
docker-compose up -d
```

Servisleri Başlatma

Tüm Servisleri Başlat
```
cd airflow_docker
docker-compose up -d
```

Servisleri Durdur
```
docker-compose down
```

Servisleri Durdur ve Verileri Sil
```
# DİKKAT: Tüm veriler silinir!
docker-compose down -v
```

Tek Servisi Yeniden Başlat
```
docker-compose restart fastapi
docker-compose restart scheduler
```

Logları İzle
```
# Tüm servisler
docker-compose logs -f

# Belirli servis
docker-compose logs -f scheduler
docker-compose logs -f fastapi
```

Pipeline Açıklaması

DAG: `ev_full_pipeline`

```
┌──────────────────────────┐
│ Step 1: Data Sync        │  Her saat çalışır
│ MariaDB → TimescaleDB    │
└──────────────────────────┘
            │
            ▼
┌──────────────────────────┐
│ Step 2: Model Train      │  H2O AutoML + Flask karşılaştırması
│ + Flask Comparison       │
└──────────────────────────┘
            │
            ▼
┌──────────────────────────┐
│ Step 3: Deploy           │  H2O > Flask ise deploy et
│ Decision                 │
└──────────────────────────┘
            │
            ▼
```

```
┌──────────────────────┐
│ Step 4: Monitoring   │  Drift detection
└──────────────────────┘
```

Step Detayları

Step	Fonksiyon	Açıklama
1	`migrate_vehicle_data() + preprocess_and_store_segments()`	MariaDB'den son 1 saatlik veriyi çeker, segmentlere ayırır, TimescaleDB'ye yazar
2	`train_h2o_automl()`	H2O AutoML ile model eğitir, Flask base model ile karşılaştırır
3	`evaluate_and_deploy_mae()`	MAE metriğine göre deploy kararı verir
4	`check_model_drift()`	Model performans sapmasını izler

Model Karşılaştırma Mantığı

```
Flask Base Model (CatBoost) MAE: 187.29
H2O AutoML Model MAE: 126.65

H2O MAE < Flask MAE → Deploy edilir
H2O MAE >= Flask MAE → Rejected, Flask kullanılmaya devam eder
```

Veritabanı Şeması

TimescaleDB Tabloları

Tablo	Açıklama
`vehicle_data`	Ham telemetri verisi
`vehicle_segments`	İşlenmiş segment verileri
`model`	Model tanımları
`model_version`	Model versiyonları (candidate/deployed/archived/rejected)
`model_run`	Eğitim geçmişi
`model_metric`	Performans metrikleri
`feature_set`	Feature tanımları
`deployment_event`	Deploy/rollback geçmişi
`prediction_log`	Tahmin logları

Model Status Akışı

```
candidate → deployed (başarılı deploy)
candidate → rejected (Flask'tan kötü)
deployed → archived (yeni model deploy edildiğinde)
```

Sorun Giderme

Airflow UI'a Erişilemiyor

```
# Container'ların çalışıp çalışmadığını kontrol et
docker-compose ps

# Webserver loglarını kontrol et
docker-compose logs webserver
```

"Database not initialized" Hatası

```
docker-compose down
docker-compose up -d postgres
sleep 30
docker-compose up -d
```

MariaDB Bağlantı Hatası

1. MariaDB'nin çalıştığından emin olun
2. `MARIADB_URI` şifresini kontrol edin
3. Linux'ta `host.docker.internal` yerine host IP kullanın:

   ```
   MARIADB_URI =
   "mysql+mysqldb://root:password@192.168.1.100:3306/fleetmanagementdb"
   ```

TimescaleDB Tablo Hatası

```
# Tabloları yeniden oluştur
Get-Content init_model_tables.sql | docker exec -i timescaledb psql -U root -
d ev_telemetry
```

FastAPI Model Yüklenmiyor

```
# Container'ı yeniden başlat
docker-compose restart fastapi

# Logları kontrol et
docker-compose logs fastapi
```

H2O Memory Hatası

`docker-compose.yaml`'da memory limitini artırın:

```
scheduler:
  deploy:
    resources:
      limits:
        memory: 8G
```

Yapılandırma

DAG Zamanlaması

`dags/ev_full_pipeline.py` dosyasında:

```
schedule_interval="@hourly"  # Her saat
# veya
schedule_interval="0 */2 * * *"  # Her 2 saatte bir
```

H2O AutoML Parametreleri

`dags/ev_h2o_train.py` dosyasında:

```
aml = H2OAutoML(
    max_runtime_secs=50,  # Eğitim süresi (saniye)
    seed=42,
    sort_metric="RMSE",
    nfolds=3,
)
```

Feature Listesi

Hem H2O hem Flask modeli şu 5 feature'ı kullanır:

```
feature_cols = [
    "segment_length",      # metre
    "slope",               # derece
    "avg_vehicle_speed",   # km/h
    "avg_Acceleration",    # m/s²
    "avg_Total_Mass",      # kg
]
```

Erişim Adresleri

Servis	URL	Varsayılan Kullanıcı
Airflow UI	http://localhost:8085	airflow / airflow
MLflow UI	http://localhost:5000	-
FastAPI Docs	http://localhost:8010/docs	-
TimescaleDB	localhost:5433	root / tsdbpass

Güvenlik Notları

Production ortamı için:

1. **Şifreleri değiştirin:**
   - PostgreSQL, TimescaleDB şifreleri

  - Airflow admin şifresi
  - MariaDB bağlantı şifresi
2. **Fernet Key oluşturun:**

```
from cryptography.fernet import Fernet
print(Fernet.generate_key().decode())
```

3. **Port'ları kısıtlayın:**

  - Sadece gerekli port'ları expose edin
  - Firewall kuralları ekleyin

Günlük Bakım

Her Gün

- Airflow UI'dan DAG durumunu kontrol edin
- Failed task'ları inceleyin

Her Hafta

- Docker disk kullanımını temizleyin:
  `docker system prune -f`
- Log dosyalarını arşivleyin

Her Ay

- Docker image'larını güncelleyin
- TimescaleDB'de eski verileri arşivleyin

Destek

Sorun yaşarsanız:

1. Bu dokümanın "Sorun Giderme" bölümüne bakın
2. Docker loglarını inceleyin
3. Airflow task loglarını kontrol edin